

INFORME TÉCNICO PROYECTO 3

BASES DE DATOS AVANZADAS

Camila Martínez
Juan Manuel Florez
Thomas Buitrago

1. Introducción

Este proyecto consistió en diseñar e implementar una solución de Business Intelligence para una empresa minorista colombiana simulada llamada Mercamax Colombia. La idea fue construir algo funcional de verdad, no solo diagramas o documentación, sino un flujo completo que va desde la base operacional hasta los tableros en Power BI.

La arquitectura implementada tiene tres capas de bases de datos más la capa de visualización:

- BI_OLTP: base operacional con los datos del negocio
- BI_Staging: zona intermedia de limpieza y validación
- BI_DW: Data Warehouse con modelo dimensional
- Power BI: tableros conectados al DW

El proyecto responde a preguntas reales de negocio como cuánto se vendió por tienda, qué categorías generan más margen, qué tan lejos están las ventas de las metas, y qué productos tienen menor inventario disponible.

2. Caso de negocio

Mercamax Colombia es una cadena minorista ficticia con diez sedes en ciudades como Bogotá, Medellín, Cali, Barranquilla, Bucaramanga, Cartagena, Pereira, Manizales, Villavicencio y Pasto. La empresa vende productos de diez categorías distintas a través de cinco canales: tienda física, marketplace, WhatsApp, app móvil y página web.

La gerencia necesita tomar decisiones sobre ventas, rentabilidad, inventario, cumplimiento de metas y desempeño por sede. Antes de este proyecto, esa información estaba dispersa en el sistema operacional y era difícil de consolidar para análisis.

3. Arquitectura general

La solución se construyó en cuatro capas. Primero el OLTP donde se registran todas las transacciones. Luego el Staging donde se copian esos datos, se limpian y se validan. Después el Data Warehouse con el modelo dimensional. Y finalmente Power BI conectado al DW.

Esta separación es importante porque tiene un propósito claro en cada etapa. El OLTP no está pensado para análisis, está optimizado para registrar operaciones rápido y mantener integridad. El DW en cambio está optimizado para leer y agregar datos, por eso usa un modelo estrella en lugar del modelo normalizado del OLTP.

4. Base de datos operacional (BI_OLTP)

La base OLTP quedó con catorce tablas. Todas tienen claves primarias, claves foráneas, restricciones CHECK y campos NOT NULL donde corresponde.

Las tablas principales son:

- Categorías, Proveedores, Productos
- Tiendas, Vendedores, Clientes, CanalVenta
- Ventas, DetalleVentas
- InventarioDiario
- Compras, DetalleCompras
- Devoluciones
- MetasComerciales

El diseño está normalizado hasta tercera forma normal. Por ejemplo, las ventas se dividen en dos tablas: Ventas que guarda el encabezado de la factura, y DetalleVentas que guarda cada producto dentro de esa factura. Esto permite analizar ventas por producto sin duplicar información del cliente o de la tienda en cada línea.

4.1 Volúmenes generados

Tabla	Registros
Categorias	10
Proveedores	20
Tiendas	10
CanalVenta	5
Clientes	1.000
Vendedores	20
Productos	200
Ventas	50.000
DetalleVentas	150.000
InventarioDiario	730.000
Compras	2.000
DetalleCompras	6.000
Devoluciones	1.000
MetasComerciales	2.400

Los datos se generaron con T-SQL usando INSERT INTO ... SELECT y CROSS JOIN, no con ciclos WHILE. Esto hace la carga mucho más rápida y evita errores de variables anidadas.

5. Fuentes externas CSV

El proyecto integra dos archivos CSV como fuentes externas, que simulan información que en una empresa real llegaría desde el área comercial o desde bodegas:

- metas_mensuales.csv: metas comerciales por año, mes, tienda y categoría
- ajustes_inventario.csv: ajustes manuales de inventario con motivo y usuario responsable

Para cargar estos archivos en SQL Server fue necesario primero crear tablas destino en BI_Staging y luego usar BULK INSERT. SQL Server no puede consultar un CSV directamente como si fuera una tabla; necesita una estructura definida primero. Las tablas creadas fueron ext_MetasMensualesCSV y ext_AjustesInventarioCSV.

6. Zona de preparación (BI_Staging)

El Staging funciona como puerta de entrada al Data Warehouse. Antes de cargar cualquier dato al DW, pasa primero por esta zona donde se revisa la calidad y se aplican transformaciones.

Las tablas de Staging son: stg_Clientes, stg_Productos, stg_Ventas, stg_Inventario, stg_Metas, stg_Devoluciones y stg_Compras. Cada una tiene tres campos de control adicionales: EsValido, MensajeValidacion y FechaCargaStaging.

6.1 Transformaciones aplicadas

En esta capa se aplicaron varias transformaciones importantes:

- Limpieza de nulos: se reemplazan por valores por defecto según el campo
- Estandarización de ciudades y departamentos: para manejar variaciones en la escritura
- Asignación de región: se calcula a partir del departamento
- Validación de claves: se verifica que cada ClientelD, ProductoID y TiendaID exista en el OLTP
- Detección de duplicados: se evita cargar el mismo registro dos veces
- Campos derivados: por ejemplo MargenUnitario en productos y UtilidadBruta en ventas

Los registros que no pasan alguna validación se marcan con EsValido = 0 y un mensaje que explica el problema. Solo los registros válidos se cargan al DW.

7. Data Warehouse (BI_DW)

El Data Warehouse usa un modelo estrella. La decisión de usar estrella en lugar de copo de nieve fue consciente: el copo de nieve normaliza más las dimensiones, lo que puede ahorrar espacio pero complica las consultas. Con el modelo estrella Power BI puede consultar directamente sin joins adicionales.

7.1 Dimensiones

Dimensión	Propósito
DimFecha	Analizar por año, trimestre, mes, semana, día y fin de semana
DimCliente	Analizar por cliente, segmento y ubicación geográfica
DimProducto	Analizar por producto, categoría y proveedor
DimTienda	Analizar por tienda, ciudad, departamento y región
DimVendedor	Analizar desempeño por vendedor
DimProveedor	Analizar compras por proveedor
DimCanalVenta	Analizar ventas por canal de venta

DimPromocion	Estructura preparada para futuras promociones
DimGeografia	Consolidar ciudad, departamento y región

7.2 Tablas de hechos

Tabla de hechos	Granularidad
FactVentas	Una línea de venta por producto, cliente, tienda, vendedor y fecha
FactInventarioDiario	Stock diario por producto y tienda
FactMetasComerciales	Meta mensual por tienda y categoría
FactDevoluciones	Una devolución asociada a una línea de venta
FactCompras	Una línea de compra por producto y proveedor

8. Justificación del modelo dimensional

8.1 Granularidad de cada tabla de hechos

FactVentas tiene granularidad de línea de venta porque necesitamos saber qué producto específico se vendió dentro de cada factura. Si guardáramos solo el total de la factura, perderíamos la capacidad de analizar ventas por categoría o por producto.

FactInventarioDiario tiene granularidad diaria por producto y tienda porque el inventario cambia todos los días y necesitamos detectar quiebres de stock en días específicos.

FactMetasComerciales tiene granularidad mensual por tienda y categoría, que es el nivel al que la empresa define sus objetivos comerciales.

FactDevoluciones tiene la misma granularidad que una línea de DetalleVentas porque cada devolución corresponde a un producto específico de una venta específica.

FactCompras tiene granularidad de línea de compra, similar a las ventas, para saber exactamente qué se compró a cada proveedor.

8.2 Medidas aditivas, semi-aditivas y no aditivas

Las medidas aditivas son las que se pueden sumar en cualquier dimensión sin problema. Por ejemplo Total Ventas, Total Costo, Utilidad Bruta, Total Compras y Total Devoluciones. Si sumas ventas por tienda, por mes o por categoría, el resultado siempre tiene sentido.

Las medidas semi-aditivas son las que se pueden sumar en algunas dimensiones pero no en todas. El inventario es el ejemplo clásico. Puedo sumar el stock de diferentes productos en una misma fecha, pero no puedo sumar el stock de un producto a través de varios días porque ese resultado no representa nada real.

Las medidas no aditivas son los porcentajes y ratios que nunca se deben sumar directamente. Margen Bruto %, Cumplimiento Meta %, Tasa Devolución % y Crecimiento Ventas % son ejemplos. Si sumas los márgenes de dos categorías no obtienes el margen combinado; tienes que recalcularlo dividiendo la utilidad total entre las ventas totales.

8.3 Por qué el inventario no se suma en el tiempo

El inventario es una fotografía de un momento, no un flujo acumulado. Si tengo 100 unidades el lunes, 95 el martes y 90 el miércoles, sumarlas da 285, pero eso no significa que haya tenido 285 unidades en algún momento ni que hayan salido 285 unidades. Por eso en Power BI usamos el promedio del inventario o el valor al final del período, no la suma.

8.4 Jerarquías dimensionales

El modelo tiene cuatro jerarquías principales que permiten hacer drill-down y roll-up en los análisis:

- Tiempo: Año → Trimestre → Mes → Día
- Geografía: Región → Departamento → Ciudad → Tienda
- Producto: Categoría → Producto
- Comercial: Tienda → Vendedor

8.5 Dimensiones con SCD tipo 2

En la versión actual las dimensiones son estáticas, pero en una implementación más completa las siguientes deberían manejar Slowly Changing Dimensions tipo 2:

- DimCliente: porque el segmento puede cambiar (de Bronce a Oro, por ejemplo)
- DimProducto: porque el precio o la categoría pueden cambiar
- DimVendedor: porque puede cambiar de tienda
- DimTienda: porque puede cambiar de región o dirección

Con SCD tipo 2 no se sobrescribe el registro anterior sino que se crea uno nuevo con fechas de vigencia, conservando el historial completo para análisis históricos.

8.6 Diferencia entre OLTP y OLAP

Aspecto	OLTP (BI_OLTP)	OLAP / DW (BI_DW)
Propósito	Registrar transacciones	Analizar información

Diseño	Normalizado (3FN)	Dimensional (estrella)
Usuario típico	Sistema operacional	Gerencia y análisis
Consulta típica	Detalle de una venta	Ventas por mes y categoría
Optimizado para	Escritura y consistencia	Lectura y agregación
Joins	Muchos y complejos	Pocos y directos

9. Procesos ETL

Los ETL se implementaron como procedimientos almacenados en T-SQL. El flujo completo es: cargar Staging desde el OLTP, transformar y validar, cargar dimensiones, cargar hechos, y registrar todo en ETL_Log.

9.1 Procedimientos implementados

- CargarDimFecha
- CargarDimCliente
- CargarDimProducto
- CargarDimTienda
- CargarDimVendedor
- CargarDimProveedor
- CargarDimCanalVenta
- CargarDimGeografia
- CargarFactVentas
- CargarFactInventarioDiario
- CargarFactMetasComerciales
- CargarFactDevoluciones
- CargarFactCompras
- ValidarCalidadDatos

9.2 Tabla ETL_Log

Cada procedimiento registra su ejecución en la tabla ETL_Log con los siguientes campos: NombreProceso, FechaInicio, FechaFin, Estado, RegistrosLeídos, RegistrosCargados, RegistrosRechazados y MensajeError.

Esto permite tener trazabilidad completa del proceso y detectar si algo falla sin tener que revisar el código manualmente.

9.3 Problema encontrado: devoluciones en enero 2025

Durante la carga de FactDevoluciones se encontró que 17 registros no cargaban. El diagnóstico fue que algunas devoluciones tenían fechas en enero de 2025 porque se generaban varios días después de ventas realizadas al final de 2024. Como DimFecha solo llegaba hasta el 31 de diciembre de 2024, esas fechas no existían en la dimensión.

La solución fue completar DimFecha con las fechas faltantes antes de cargar FactDevoluciones. No se eliminaron ni forzaron los registros porque las devoluciones eran válidas; el problema era la dimensión, no el hecho. Después de esta corrección FactDevoluciones quedó con los 1.000 registros esperados.

10. Modelo y dashboard en Power BI

Power BI se conectó directamente al Data Warehouse BI_DW en SQL Server. El modelo semántico replica las relaciones del DW con dimensiones y hechos. Se marcó DimFecha como tabla de fechas para habilitar las funciones de time intelligence en DAX.

10.1 Páginas del dashboard

El dashboard quedó con cinco páginas:

- Resumen Ejecutivo: KPIs principales (Total Ventas, Utilidad Bruta, Margen %, Cumplimiento Meta %, Ticket Promedio), ventas por mes, por categoría y top tiendas
- Análisis de Ventas: ventas por vendedor, canal, segmento, categoría, tienda y región, con tabla de detalle
- Inventario: inventario promedio, entradas, salidas, stock mínimo, productos con menor inventario y rotación en unidades
- Metas y Operación: ventas reales vs metas, cumplimiento por tienda, brecha, devoluciones por motivo y compras por proveedor
- Rentabilidad y OLAP: ventas, costos, utilidad y margen por categoría y tienda, con matriz OLAP con jerarquías de tiempo, región y categoría

10.2 Medidas DAX implementadas

Se crearon 23 medidas DAX, superando el mínimo de 15 requerido. Las principales son:

Medida	Descripción
Total Ventas	SUM de ValorVenta en FactVentas
Total Costo	SUM de CostoTotal en FactVentas
Utilidad Bruta	Total Ventas menos Total Costo
Margen Bruto %	DIVIDE(Utilidad Bruta, Total Ventas)
Ventas Año Anterior	CALCULATE con SAMEPERIODLASTYEAR

Crecimiento Ventas %	Variación entre año actual y anterior
Total Meta	SUM de ValorMeta en FactMetasComerciales
Cumplimiento Meta %	DIVIDE(Total Ventas, Total Meta)
Brecha Meta	Total Ventas menos Total Meta
Ranking Productos	RANKX con ALL sobre DimProducto
Ranking Tiendas	RANKX con ALL sobre DimTienda
Promedio Movil Ventas 3 meses	AVERAGEX con DATESINPERIOD
Participacion % por Categoria	DIVIDE con CALCULATE y ALL
Ticket Promedio	DIVIDE(Total Ventas, DISTINCTCOUNT facturas)
Inventario Promedio	AVERAGE de StockFinal
Rotacion de Inventario Unidades	Unidades Vendidas / Inventario Promedio
Entradas Totales	SUM de Entradas en FactInventarioDiario
Salidas Totales	SUM de Salidas en FactInventarioDiario
Stock Minimo	MIN de StockFinal
Total Devoluciones	SUM de ValorDevuelto
Tasa Devolucion %	DIVIDE(Total Devoluciones, Total Ventas)
Total Compras	SUM de ValorCompra en FactCompras
Unidades Compradas	SUM de Cantidad en FactCompras

10.3 Decisión sobre el cumplimiento de metas

El cumplimiento de metas quedó en 12.15%, lo cual se ve bajo. Esto ocurrió porque las metas generadas eran altas frente al volumen de ventas sintéticas. Se evaluó la posibilidad de ajustar las metas para que el indicador se viera más razonable visualmente, pero se decidió no hacerlo.

La razón es que en BI los tableros no deben maquillar resultados. Si hay una brecha, el análisis debe mostrarla y el equipo directivo debe decidir qué hacer. Modificar los datos para que el dashboard se vea bien habría desvirtuado el propósito del análisis.

11. Despliegue en Azure

Además de la ejecución local, el proyecto fue replicado en una máquina virtual de Azure. La VM se llama vm-proyecto-bi-retail, usa Windows Server con SQL Server Developer, y tiene 2 vCPUs y 8 GB de RAM.

El proceso fue el siguiente: se creó la VM, se instaló SQL Server y SSMS, se descargó el repositorio desde GitHub dentro de la VM, se copiaron los archivos CSV a la ruta esperada por BULK INSERT, y se ejecutaron todos los scripts SQL en orden. Se validó que las tres bases quedaran creadas correctamente y que los conteos coincidieran con los de la ejecución local.

Los datos de acceso a la VM (IP pública, usuario y contraseña) se entregan por canal privado al profesor y no se publican en el repositorio por razones de seguridad.

12. Ética y uso de inteligencia artificial

El proyecto usó IA como herramienta de apoyo durante el desarrollo. Es importante ser claro sobre en qué consistió ese apoyo y qué parte del trabajo fue hecha directamente por el equipo.

La IA se usó principalmente para generar borradores de scripts SQL, proponer medidas DAX, revisar errores de sintaxis y ayudar a organizar la documentación. Sin embargo, aceptar automáticamente lo que genera la IA no funciona en un proyecto como este, porque los scripts requieren ajustes, los errores que aparecen son específicos del ambiente, y muchas decisiones dependen del contexto del proyecto.

12.1 Qué hizo la IA

- Generar borradores iniciales de scripts de creación de tablas
- Proponer estructuras de procedimientos ETL
- Sugerir medidas DAX y sus fórmulas
- Apoyar en la redacción inicial de documentación
- Revisar errores de sintaxis en T-SQL

12.2 Qué hizo el equipo

- Ejecutar todos los scripts en SQL Server y validar resultados
- Corregir errores reales que aparecieron durante la ejecución
- Tomar decisiones de diseño como la granularidad de cada hecho
- Resolver el problema de devoluciones con fechas fuera de DimFecha
- Corregir la medida de rotación de inventario que mezclaba unidades
- Decidir no ajustar las metas para no manipular el análisis
- Construir los tableros en Power BI y ajustar las visualizaciones
- Crear la VM en Azure y replicar el proyecto en nube
- Organizar el repositorio y las evidencias

Una estimación razonable es que la IA apoyó entre el 55% y el 65% en generación inicial de código y documentación. El resto fue implementación real, pruebas, corrección de errores, toma de decisiones y montaje en producción.

El uso de IA se declara con transparencia porque el objetivo del proyecto es aprender a construir una solución BI, no solo tener un repositorio completo. Las decisiones de arquitectura, granularidad y diseño deben entenderse y poder defenderse independientemente de quién generó el código inicial.

13. Conclusiones

El proyecto permitió construir una solución BI completa y funcional. Lo más valioso no fue que el código corriera, sino entender por qué cada decisión de diseño importa.

Por ejemplo, entender que el inventario no se suma en el tiempo no es un detalle técnico menor; es una diferencia conceptual que cambia completamente cómo se interpreta ese dato en un tablero ejecutivo. Lo mismo aplica para la granularidad: si FactVentas hubiera quedado a nivel de factura en lugar de línea de venta, no habría sido posible responder preguntas básicas del negocio como cuánto se vendió de cada categoría.

Otro aprendizaje importante fue sobre la calidad de datos. El Staging existe precisamente porque en proyectos reales los datos nunca llegan perfectos. Tener una capa de validación antes de cargar el DW protege la integridad del análisis.

En cuanto a Power BI, la mayor dificultad no fue crear las visualizaciones sino asegurarse de que las medidas DAX fueran conceptualmente correctas. Una medida que técnicamente funciona pero que conceptualmente suma cosas que no se deben sumar es peor que no tener la medida.

Finalmente, el proyecto dejó claro que una solución BI no es solo tecnología. Requiere entender el negocio, tomar decisiones de diseño conscientes, y saber interpretar los resultados aunque no sean los que uno esperaba ver.